

The Chase Is the Curriculum, the Capture Anchors the Credit: Pursuit-Evasion Self-Play for Zero-Data LLM Reasoning

Jing Yu¹, Shengchao Chen^{2*}, Yiyun Tan¹

¹College of Computer and Information Engineering, Xi'an University of Architecture and Technology

²Australian Artificial Intelligence Institute, University of Technology Sydney
yujing1203@xauat.edu.cn, shengchao.chen.uts@gmail.com, yytanmax@gmail.com

Abstract

Reinforcement learning with verifiable rewards has become the dominant recipe for improving large language model reasoning, yet it presumes large human-curated task collections. Zero-data self-play removes this dependency, but existing methods vet learnability only by probing candidates and rejecting post hoc, never learning where along an environment’s difficulty axis to place a task, and credit the solver with sparse terminal rewards alone. We recast zero-data self-play as a *pursuit-evasion game*: in **LURE**, an LLM *evader* positions tasks along each environment’s difficulty axis to stay one step ahead of a planner-executor *pursuer* that hunts it down through verifiable interaction. The evader is trained on a *capture-frontier* reward that peaks when the solver captures it on exactly half of its rollouts, turning barely catchable into a learned positioning strategy rather than a hand-tuned rejection band. The pursuer earns *capture-anchored dense process credit*, in which monotone verifier progress is group-normalized jointly with the terminal capture under a round-anchored KL that keeps the co-evolution stable. Across three verifiable reasoning environments and three backbone families, **LURE** outperforms advanced baselines under unified/specialist settings, while the unified model attains stronger aggregate OOD zero-shot accuracy than all trained baselines across nine held-out benchmarks from three task families.

Introduction

Large language models are increasingly deployed on multi-step verifiable reasoning tasks, including multi-hop question answering (Gong et al. 2025), instruction following (Zhou et al. 2023), and logic-grid deduction (Lin et al. 2025), where success requires planning and executing several intermediate steps. The prevailing route to such abilities is reinforcement learning with verifiable rewards (RLVR), which optimizes the policy directly against programmatic checkers rather than a learned reward model (Shao et al. 2024; Guo et al. 2025; Lambert et al. 2024). Its effectiveness, however, is bounded by the supply of human-curated tasks with gold verification, which is expensive to build and quickly saturated by strong models (Fang et al. 2026). Self-evolving training promises to lift this ceiling: the model generates its own curriculum and improves from verifiable feedback with *zero* external data (Huang et al. 2025; Zhao et al. 2026).

Existing zero-data self-play, whether grounded in majority-vote pseudo-labels (Huang et al. 2025) or a code

executor (Zhao et al. 2026) (**Fig. 1a,b**), gives its curriculum no explicit difficulty axis: a proposer authors candidate tasks in text space, and learnability is enforced only *post hoc*, by probing each candidate with solver rollouts and rejecting those whose empirical solve rate falls outside a fixed band (Huang et al. 2025). Task placement therefore stays implicit in the authored text, and the hand-set band edges never adapt as the solver’s competence changes. When that proposal is mismatched with the solver, rejection sampling degenerates into paying rollout cost only to discard almost everything.

The same non-adaptive curriculum leaves the solver underinformed. A multi-step episode returns a single terminal reward, so the trajectory is credited as one block, and under group-relative policy optimization the gradient vanishes whenever every rollout of a task fails (Prakash and Buvaresh 2025), precisely the case for hard, self-generated tasks. Densifying this signal with a process reward model reintroduces the external supervision that self-play is meant to eliminate (Lightman et al. 2024; Wang et al. 2024), and the round-by-round drift of the task distribution makes optimization non-stationary and prone to collapse. What is missing is a loop that learns from the verifier alone: a challenger trained to place tasks at the solver’s moving frontier, and dense credit derived, without annotation, from the environment’s own progress signal, stabilized across co-evolution.

To address these challenges, this paper formulates zero-data self-play as a *pursuit-evasion game* (**Fig. 1c**), reminiscent of Scotland Yard and of cops-and-robbers games on graphs (Bonato 2011), in which an LLM challenger (the evader) learns where to *position* tasks and a planner-executor solver (the pursuer) learns to hunt them down, co-evolving on verifiable natural-language reasoning environments. The evader is optimized with GRPO on a *capture-frontier* reward that peaks when the solver captures a task on half of its rollouts, minus a difficulty-signature repetition penalty, so “stay exactly one step ahead of the pursuer” becomes a positioning strategy the challenger *learns*, rather than a hand-tuned rejection band applied after blind sampling. In turn, the pursuer is optimized with *capture-anchored dense process credit*, in which each executor step earns the monotone verifier-progress it adds, with zero-progress steps paying a small redundancy cost, jointly group-normalized with the terminal capture, under a round-anchored KL applied to both players so that neither escapes the other’s reach within

*Corresponding author.

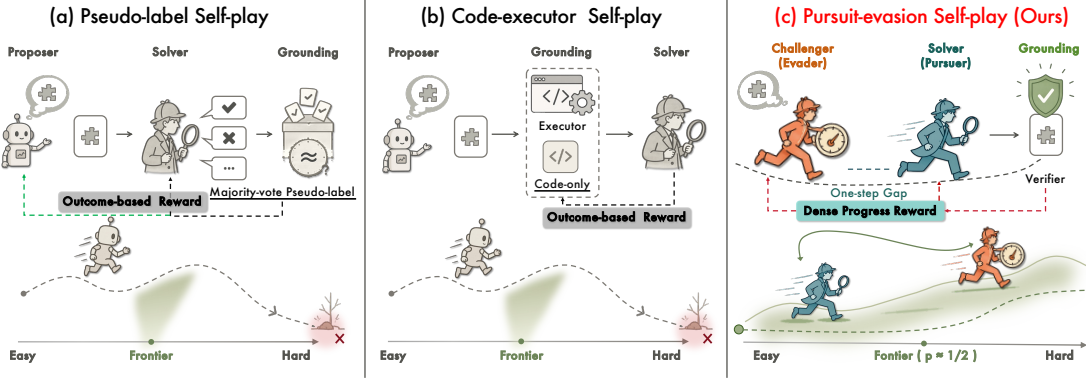


Figure 1: Zero-data self-play paradigms. (a) Pseudo-label self-play uses majority-vote labels (Huang et al. 2025), and (b) code-executor self-play uses program execution (Zhao et al. 2026). Both author tasks free-form, vetted post hoc under outcome-only supervision. (c) **LURE** adapts the challenger toward solver’s capture frontier and gives the pursuer dense verifier-based credit.

a round. Instantiated over three heterogeneous environments with per-role weights shared across all of them, this chase turns verifier feedback into both players’ training signal without any human data. We call the method **LURE**: the evader acts as a learned lure, holding each task barely catchable so that it pulls the solver up to its own frontier rather than fleeing beyond reach. Our contributions are fourfold:

- We formulate zero-data self-play as a pursuit-evasion game and, to our knowledge, are the first to *co-train* an LLM difficulty-selection challenger with a multi-turn planner-executor solver on verifiable NL reasoning.
- We propose a capture-frontier challenger objective that turns the solver’s group capture statistics into a training signal for difficulty positioning, replacing post-hoc rejection with a learned curriculum policy.
- We introduce a capture-anchored dense process credit from the environment verifier alone, with round-anchored KL stabilization applied to both players.
- Across three verifiable reasoning environments and backbone families, **LURE** outperforms advanced baselines under unified/specialist settings, attaining stronger aggregate OOD zero-shot accuracy across nine benchmarks.

Related Work

Verifiable Multi-Step Reasoning. Multi-step reasoning has advanced by making intermediate computation explicit and verifiable. Chain-of-thought exposes reasoning paths (Wei et al. 2022), while step-level scoring improves both training and inference beyond final-answer supervision (Lightman et al. 2024). Reinforcement learning with verifiable rewards turns programmatic checkers into optimization targets, with group-relative policy optimization now standard for reasoning LLMs (Shao et al. 2024; Guo et al. 2025; Sheng et al. 2024). For multi-step tasks, agents interleave decomposition, grounded action, and self-correction (Yao et al. 2022; Shinn et al. 2023), while process reward models densify terminal rewards but typically require step annotations that zero-data methods seek to avoid (Wang et al. 2024). These advances also motivate programmatically

verifiable benchmarks for instruction following (Zhou et al. 2023), logic-grid deduction (Lin et al. 2025), and multi-hop relational question answering (Gong et al. 2025). This line of work improves *solving* but leaves task generation open.

LLM Self-Evolving. Self-evolving training lets models generate their own curricula by bootstrapping supervision through synthesized instructions (Wang et al. 2023) or outcome-filtered rationales (Zelikman et al. 2022), or by framing training as a game between a proposer and a solver (Liu et al. 2025; Zhao et al. 2026). Most closely related, R-Zero co-evolves an LLM challenger that authors problem text, keeping candidates only when solve rates fall within a fixed hand-set band (Huang et al. 2025), which costs a probe of solver rollouts per candidate. Separately, outcome-only group-relative training provides no learning signal when an entire group fails, known as the zero-reward barrier (Prakash and Buvanesh 2025). Beyond language models, adaptive task generation at the learner’s frontier appears in asymmetric self-play (Sukhbaatar et al. 2017), goal generators targeting intermediate success rates (Florensa et al. 2017), and regret-based environment design (Dennis et al. 2020). Our setting instead considers natural-language tasks generated by an LLM proposer and supervised solely by a programmatic verifier. Building on the pursuit-evasion framing (Bonato 2011), **LURE** trains the evader with a capture-frontier reward and the pursuer with dense verifier progress, using the same programmatic signal for both roles.

Methodology

Problem Formulation. We consider a family of verifiable language reasoning environments $\mathcal{E}$. Each environment e provides a programmatic task generator $g_e(d, \omega)$ indexed by a difficulty parameter $d \in [0, 1]$, a multi-turn interaction protocol, and a verifier that emits two signals per trajectory τ : a binary terminal *capture* $R(\tau) \in \{0, 1\}$, which fires only when the task is solved in full, and a running progress $\phi_t \in [0, 1]$, the best verified progress so far, with $\phi = 1$ at capture. We write $\Delta\phi_t$ for the nonnegative increment in best verified progress. We cast self-play as a two-player pursuit-

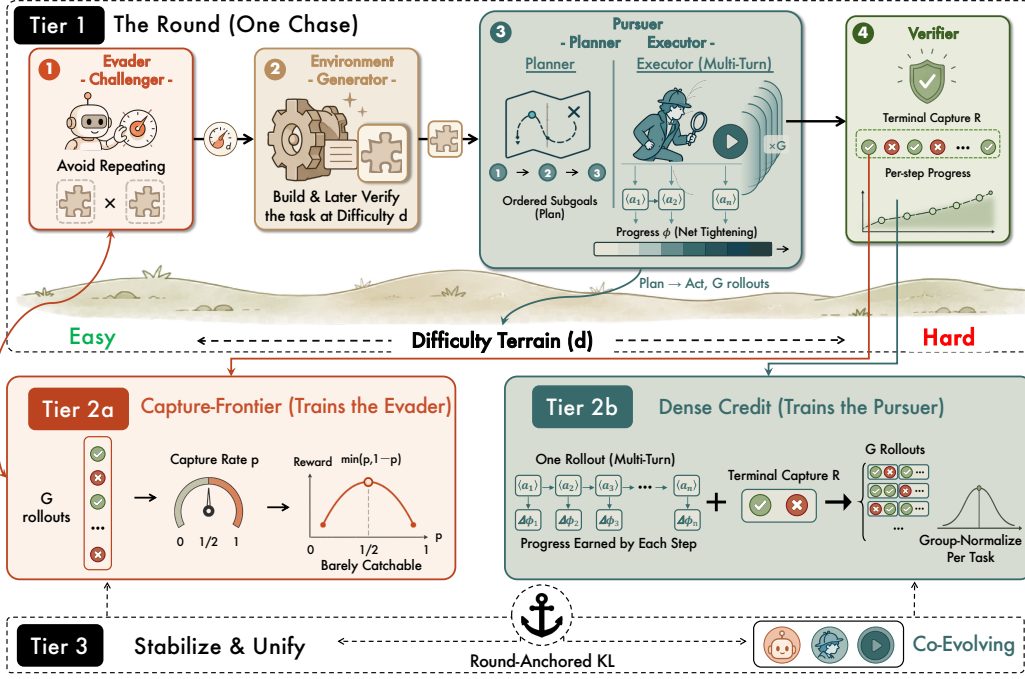


Figure 2: **Overview of LURE.** *Tier 1:* the evader selects a difficulty d with a penalty for repeated signatures, the generator instantiates the task, the planner-executor pursuer runs G multi-turn rollouts, and the verifier returns capture R and stepwise progress ϕ . *Tier 2a:* the evader optimizes $\min(p, 1-p)$, peaking at the capture frontier $p = 1/2$. *Tier 2b:* the pursuer receives task-wise group-normalized credit from R and $\Delta\phi$. *Tier 3:* a round-anchored KL stabilizes all three policies.

evasion interaction: the *evader* π_C selects where on each environment’s difficulty axis to place the next tasks, which the generator instantiates, and the *pursuer* $\pi_S = (\pi_{\text{plan}}, \pi_{\text{exec}})$ is a planner-executor pair that hunts each task down, with empirical capture rate on task x over G rollouts

$$p(x) = \frac{1}{G} \sum_{g=1}^G \mathbb{1}[R(\tau_g) = 1]. \quad (1)$$

The two players optimize a coupled pair of objectives, stated as a pair rather than a minimax and carrying no equilibrium or convergence claim:

$$\begin{aligned} \max_{\pi_C} \mathbb{E}_x [\min(p(x), 1-p(x)) - \rho(x)], \\ \max_{\pi_S} \mathbb{E}_{x \sim \pi_C} \mathbb{E}_{\tau} \left[R(\tau) + \lambda \sum_t (\Delta\phi_t - c_{\text{red}} \mathbb{1}[\Delta\phi_t=0]) \right]. \end{aligned} \quad (2)$$

Both objectives are functions of the verifier alone, with no human-curated tasks, labels, or reward models.

Overview. Fig. 2 shows the workflow of LURE. A single round of the chase couples three co-evolving policies, the evader, the planner, and the executor, instantiated as three separate models. Each round opens by freezing a snapshot of all three, and those snapshots generate every sample the round collects. First, the evader emits K candidate tasks per environment, each placed at a target difficulty, and the well-formed ones become the round’s curriculum. Second,

the pursuer attacks each task with G plan-execute rollouts while the verifier records, per trajectory, the terminal capture R and the per-step progress $\{\phi_t\}$. Third, capture statistics reposition the evader near the pursuer’s frontier, capture and progress increments credit the executor, and the progress its plans realize credits the planner. Finally, each policy takes a single GRPO update on this round’s data. Because every trajectory was generated by the round-start snapshots, the round is simultaneous-move, and each player faces the others’ updates only in the next round.

Frontier-Seeking Evader. The evader learns *where to run* rather than what the tasks contain, repositioning along the difficulty axis against the pursuer’s measured competence. Each emitted task x receives the capture-frontier reward

$$r_C(x) = \begin{cases} \min(p(x), 1-p(x)) - \rho(x), & x \in \mathcal{V}, \\ -1 - \rho(x), & x \notin \mathcal{V}, \end{cases} \quad (3)$$

where $\mathcal{V}$ is the set of well-formed emissions the pursuer attacked, and the repetition penalty $\rho(x) = (n_{\text{sig}}(x) - 1)/N$ grows with the number of the round’s N emissions that share x ’s signature, so the evader cannot collapse onto one signature. Signatures are coarse and per-environment: hop count and population for multi-hop question answering, constraint count for instruction following, grid dimensions for logic-grid deduction. The reward peaks at $p = \frac{1}{2}$, so the evader’s optimum is to remain *barely catchable*, neither escaping ($p=0$) nor trivially caught ($p=1$). This is also where the capture signal is most informative, since binary cap-

ture gives the terminal reward a within-group variance of $p(1-p)$, and a task off the frontier returns near-identical outcomes across its group and a vanishing group-normalized advantage. Rejection filtering, by contrast, discards finished candidates but never steers where the next ones are placed. Evader advantages are group-normalized over the samples sharing a prompt slot. The generation prompt also carries an initial difficulty hint and a per-difficulty summary of pursuer competence, so this scaffolding shapes the frontier behavior alongside the reward.

Dense-Credit Pursuer. Terminal capture is sparse: on hard self-generated tasks, whole rollout groups fail and a purely terminal group-relative signal vanishes. The pursuer’s dense credit therefore comes from the verifier’s own step progress. For each executor turn t of trajectory τ we form a single raw scalar

$$r_t = R(\tau) \mathbb{1}[t = T_\tau] + \lambda (\Delta\phi_t - c_{red} \mathbb{1}[\Delta\phi_t = 0]), \quad (4)$$

and standardize these scalars jointly over the group $\mathcal{G}(x)$ of all executor turns across the G rollouts of x :

$$A_t = z_{\mathcal{G}(x)}(r_t). \quad (5)$$

Three design choices matter. First, the terminal reward is credited once, on the last executor turn, since replicating it across turns would bias the group statistics toward long trajectories. Second, the zero-progress cost is charged only when $\Delta\phi_t = 0$: an unconditional constant would be exactly cancelled by the shift invariance of group-normalization, so the indicator separates stalling from progressing whenever a group contains both. Third, because ϕ_t is best-so-far, $\Delta\phi_t \geq 0$ and $\sum_t \Delta\phi_t$ telescopes to the trajectory’s peak verified progress, so a regression receives zero dense credit rather than a penalty. When an entire group fails, the terminal term is identically zero and the dense term still carries gradient wherever step progress varies within the group. The planner receives its own credit: each rollout is scored by the stage potentials Φ_i of the executor segments it governed,

$$r^{\text{plan}}(\tau) = \sum_i \Phi_i, \quad (6)$$

group-normalized over the same task group with no terminal term, so a plan is judged by how far its execution advanced rather than by whether capture occurred.

Stabilized Co-Evolution. The chase is non-stationary by construction. The task distribution the pursuer faces, and the pursuer the evader probes, both shift from round to round, so an update regularized toward a stale reference is pulled away from the distribution that produced its data. We therefore formulate the round as a proximal co-evolution step: the evader, planner, and executor each take a single GRPO update under a KL penalty to their own round-start snapshot. That snapshot acts as behavior policy, KL reference, and proximal center, aligning the regularizer with the distribution that generated the samples and limiting within-round drift. Without it we observed format and mode collapse. Re-anchoring at the next round preserves that control while letting the joint policy trajectory adapt across rounds.

Algorithm 1 One round of LURE. **Alg. 2** in **App. A** writes out every step.

Require: policies $\pi_C, \pi_{\text{plan}}, \pi_{\text{exec}}$, environments $\mathcal{E}$, candidates K , rollouts G

Ensure: updated policies $\pi_C, \pi_{\text{plan}}, \pi_{\text{exec}}$

- 1: round-start snapshots $\bar{\pi}_C, \bar{\pi}_{\text{plan}}, \bar{\pi}_{\text{exec}} \leftarrow \pi_C, \pi_{\text{plan}}, \pi_{\text{exec}}$
 - 2: buffers $\mathcal{D}_C, \mathcal{D}_{\text{plan}}, \mathcal{D}_{\text{exec}} \leftarrow \emptyset$, task pool $\mathcal{X} \leftarrow \emptyset$
 - 3: **for** each environment $e \in \mathcal{E}$ **do**
 - 4: $\mathcal{X} \leftarrow \mathcal{X} \cup \{\text{valid tasks from } K \text{ of } \bar{\pi}_C \text{ through } g_e\}$
 - 5: **end for**
 - 6: **for** each task $x \in \mathcal{X}$ **do**
 - 7: roll out G trajectories with $(\bar{\pi}_{\text{plan}}, \bar{\pi}_{\text{exec}})$, scored by the verifier
 - 8: $p(x) \leftarrow \frac{1}{G} \sum_{g=1}^G \mathbb{1}[R(\tau_g)=1]$
 - 9: add the evader sample for x to $\mathcal{D}_C$ (Eq. 3)
 - 10: add the executor turns to $\mathcal{D}_{\text{exec}}$ (Eq. 4)
 - 11: add the planner turns to $\mathcal{D}_{\text{plan}}$ (Eq. 6)
 - 12: **end for**
 - 13: update π_C by GRPO on $\mathcal{D}_C$ with KL anchor $\bar{\pi}_C$
 - 14: update π_{plan} by GRPO on $\mathcal{D}_{\text{plan}}$ with KL anchor $\bar{\pi}_{\text{plan}}$
 - 15: update π_{exec} by GRPO on $\mathcal{D}_{\text{exec}}$ with KL anchor $\bar{\pi}_{\text{exec}}$
 - 16: **return** $\pi_C, \pi_{\text{plan}}, \pi_{\text{exec}}$
-

Optimization Objective. All three roles are trained with the same clipped GRPO objective (Shao et al. 2024; Schulman et al. 2017) and round-anchored KL, differing only in their advantages. Let $\bar{\pi}_r$ denote the round-start snapshot of role r and $\varrho_r = \pi_{\theta_r}(y | s) / \bar{\pi}_r(y | s)$ the token-level ratio to it. Each role maximizes the objective below:

$$\mathcal{J}_r(\theta_r) = \mathbb{E}[\min(\varrho_r A^r, \text{clip}(\varrho_r, 1-\epsilon, 1+\epsilon) A^r)] - \beta \text{KL}(\pi_{\theta_r} \parallel \bar{\pi}_r), \quad (7)$$

for $r \in \{C, \text{plan}, \text{exec}\}$. Each role’s advantage is its group-normalized reward: $A^C = z_{\text{slot}}(r_C)$ for the evader, $A_i^{\text{exec}} = z_{\mathcal{G}(x)}(r_t)$ for the executor, and $A_\tau^{\text{plan}} = z_{\mathcal{G}(x)}(r^{\text{plan}}(\tau))$ for the planner, with every token carrying the advantage of its turn or rollout. **Alg. 1** shows the workflow.

Theoretical Analysis. Three exact properties of the design, proved in **App. D**: the evader’s optimum sits precisely where the pursuer’s learning signal peaks, the repetition penalty makes curriculum collapse impossible, and the dense credit can never outrank capture.

Theorem 1 (Frontier signal). *Let p be a task’s capture probability and $A_1, \dots, A_G$ the group-normalized advantages of its $G \geq 2$ binary capture outcomes. Then $\sum_g A_g^2 = (G-1)\mathbb{1}[s(R) > 0]$ pointwise, with $s(R)$ the sample standard deviation of the outcomes, so $\mathbb{E}[\sum_g A_g^2] = (G-1)(1-p^G - (1-p)^G)$, uniquely maximized at $p = 1/2$, which is also the unique maximizer of r_C .*

Proposition 2 (No signature collapse). *In the continuous relaxation the maximizer of Eq. 3 over signature distributions is unique, with $m_s^* = 1/k + (f_s - f)/2$ on its support S^* of size k . Hence $k \geq 2$ and $|m_s^* - m_{s'}^*| \leq 1/4$ on S^* .*

Proposition 3 (Capture dominance). *Best-so-far progress gives $\sum_t \Delta\phi_t = \phi_{T_\tau}$, so Eq. 4 totals $R(\tau) + \lambda\phi_{T_\tau}$ –*

Method	Unified model				Per-environment specialists			
	PhantomWiki	IFEval	ZebraLogic		PhantomWiki	IFEval	ZebraLogic	
	SUCCESS $\uparrow$	Prompt Acc. $\uparrow$	Puzzle Acc. $\uparrow$	Cell Acc. $\uparrow$	SUCCESS $\uparrow$	Prompt Acc. $\uparrow$	Puzzle Acc. $\uparrow$	Cell Acc. $\uparrow$
Qwen2.5-7B-Instruct (Qwen et al. 2025)								
Base	25.0	63.0	11.6	42.3	25.0	63.0	11.6	42.3
GRPO (Shao et al. 2024)	43.3	60.6	<u>15.4</u>	<u>45.8</u>	<u>60.8</u>	<u>62.8</u>	<u>16.1</u>	<u>46.5</u>
GRPO-Zero	37.5	58.4	13.9	44.2	0.0	54.9	12.7	43.1
R-Zero (Huang et al. 2025)	<u>50.8</u>	57.9	14.2	43.6	56.7	60.1	15.0	44.8
LURE (Ours)	65.8	<u>61.2</u>	16.9	47.2	80.0	63.0	18.3	48.6
Llama-3.1-8B-Instruct (Grattafiori et al. 2024)								
Base	0.0	37.5	12.8	13.7	0.0	37.5	12.8	13.7
GRPO (Shao et al. 2024)	0.8	38.8	14.1	<u>27.9</u>	2.5	<u>43.4</u>	<u>13.6</u>	27.2
GRPO-Zero	0.8	39.4	13.0	26.4	0.8	39.4	12.9	26.2
R-Zero (Huang et al. 2025)	<u>3.3</u>	<u>42.5</u>	<u>14.6</u>	26.8	<u>4.2</u>	43.1	<u>13.6</u>	<u>28.3</u>
LURE (Ours)	4.2	44.5	15.9	29.5	5.0	44.7	15.2	29.9
Gemma-2-9B-it (Team et al. 2024)								
Base	11.7	60.4	13.1	38.4	11.7	60.4	13.1	38.4
GRPO (Shao et al. 2024)	27.5	<u>64.0</u>	<u>15.7</u>	40.2	<u>40.0</u>	62.5	<u>15.7</u>	40.3
GRPO-Zero	15.0	61.2	14.3	39.9	20.8	60.4	14.3	39.6
R-Zero (Huang et al. 2025)	<u>34.2</u>	62.5	15.1	<u>41.6</u>	30.0	<u>63.6</u>	15.0	<u>40.9</u>
LURE (Ours)	40.0	66.2	18.2	43.5	42.5	66.2	17.6	43.1

Table 1: Main results (%) on PhantomWiki, IFEval, and ZebraLogic across two settings. **Best** and second best per backbone.

$\lambda c_{red} Z(\tau)$ over a trajectory, with Z its stalled-turn count. If $\lambda c_{red}(T_{max} - 1) < 1$, every capturing trajectory strictly outranks every non-capturing one.

Experiments

Implementation Details. We used Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct, or Gemma-2-9B-it as our backbone models. All models were trained with verl-GRPO (Sheng et al. 2024; Shao et al. 2024) under FSDP-2 on $16 \times$ RTX 6000 Pro Blackwell 96 GB GPUs for $R = 8$ rounds, using no external task data. Each round emits 48 candidate tasks per environment (8 prompt slots $\times$ 6 samples) and attacks every surviving task with a group of $G = 8$ pursuer rollouts. **LURE** sets the dense-credit weight $\lambda = 0.25$ and zero-progress cost $c_{red} = 0.05$, and runs the stabilized recipe on all three policies: round-anchored KL with $\beta = 0.1$, learning rate 5×10^{-6} , and a single PPO epoch. Baselines use the optimizer settings from their own papers. The held-out protocol, covering prompts, verifiers, and decoding, is identical across methods, with full configurations, role prompts, and baseline constructions in **App. A**.

Benchmarks & Metrics. To evaluate multi-step verifiable reasoning, the setting the curriculum targets, we use three environments that carry their own programmatic verifier and need no human labels: **PhantomWiki** (Gong et al. 2025) for relational inference, **IFEval** (Zhou et al. 2023) for instruction following, and **ZebraLogic** (Lin et al. 2025) for logical deduction. Each doubles as a benchmark and is measured on a held-out split. PhantomWiki is multi-hop kinship question answering, scored as chain-verified exact match on $n = 120$, where every hop must resolve and the first error ends the chain. IFEval is scored as per-prompt accuracy on its 541 prompts under strict evaluation. ZebraLogic is scored on the official set of 1000 puzzles, as puzzle accuracy, which demands every cell of the grid, and as cell accuracy, with every metric formalized in **App. B**.

Main Results. We compare **LURE** with *Base* (zero-shot), *GRPO* trained on curated tasks, *GRPO-Zero* trained on a self-generated pool frozen after the first round, a non-adaptive curriculum, and *R-Zero* (Huang et al. 2025). **Table 1** reports results across three backbone families. Our proposed **LURE** outperforms all trained baselines in all three environments. On PhantomWiki, it exceeds R-Zero by 15.0 points and the frozen-pool control by an even larger margin. Since the frozen-pool control also improves over Base, self-generated data explains part of the gain, while the residual advantage over post-hoc band rejection reflects the contribution of adaptive task placement. On IFEval with the Qwen backbone, Base remains the strongest overall entry, and no self-play method surpasses the corrected zero-shot baseline. On ZebraLogic, **LURE** achieves smaller leads on both metrics. The larger separation reported previously resulted from gold-answer leakage rather than a genuine method difference.

Ablation Study. **Table 2** removes one component at a time, with each row’s exact configuration specified in **App. C**, and every removal lowers the mean, so each contributes to the full system. The magnitudes group by role: the three credit terms are the three smallest degradations, while every curriculum and stabilization ablation costs more than any of them, and the learned challenger is the largest at -10.6 . The asymmetry follows from what each part controls, since the challenger decides which tasks carry a usable gradient at all, whereas the credit terms only reshape a gradient it has already supplied. Stabilization patterns with the curriculum rather than with the credit terms, which is what a mechanism protecting the loop from its own non-stationarity should do. Since the components interact through one shared verifier signal, these are contributions within the full system rather than independent effects.

Hyperparameter Sensitivity. **Fig. 3** evaluates the six method-specific hyperparameters: the dense-credit weight λ , zero-progress cost c_{red} , round-anchored KL coefficient β ,

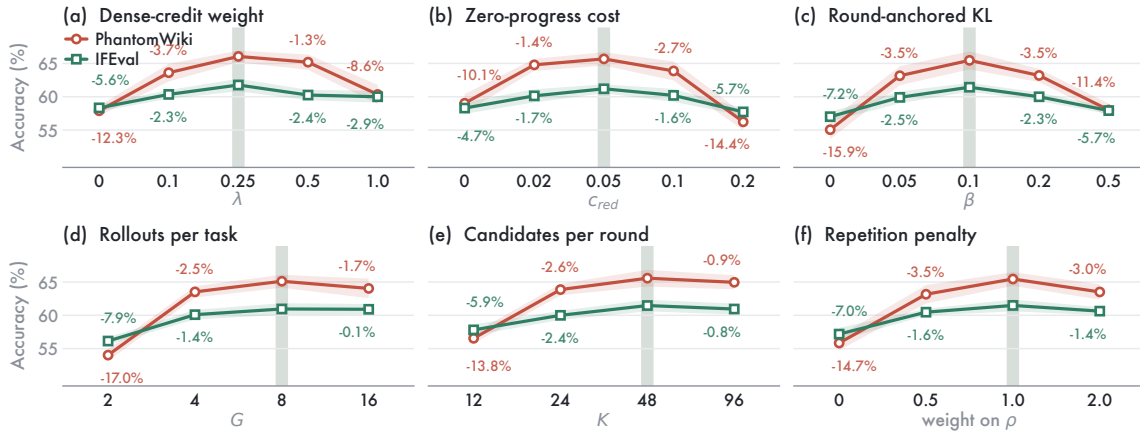


Figure 3: Hyperparameter sensitivity on Qwen2.5-7B unified model. Labels give each point’s relative drop from its curve’s best.

Configuration	PhantomWiki	IFEval	ZebraLogic	Mean	Δ
Full	65.8	61.2	16.9	48.0	—
<i>Curriculum</i>					
w/o learned challenger	40.0	58.0	14.1	37.4	−10.6
w/o capture-frontier reward	53.3	58.0	15.4	42.2	−5.8
w/o repetition penalty	55.8	56.9	14.6	42.4	−5.6
<i>Pursuer credit</i>					
w/o dense credit	57.5	57.9	15.1	43.5	−4.5
w/o zero-progress cost	59.2	58.2	15.8	44.4	−3.6
w/o planner credit	61.7	59.5	16.2	45.8	−2.2
<i>Stabilization</i>					
w/o round-anchored KL	55.0	56.7	13.4	41.7	−6.3
w/o stabilized recipe	52.5	55.5	12.9	40.3	−7.7

Table 2: Ablation on the Qwen2.5-7B-Instruct unified model. Δ is the change in mean against the full system.

rollout group size G , candidate budget K , and the repetition-penalty scale, the coefficient on $\rho(x)$ that Eq. 3 fixes to one. Performance is most sensitive to β and G . Removing the round-anchored KL substantially reduces accuracy, while an excessively large β also degrades performance, indicating a trade-off between stabilizing the co-evolving policies and preserving sufficient update flexibility. Reducing G similarly causes a marked decline. Because the capture rate is estimated from G rollouts, a small group yields a coarse estimate of the capture frontier and provides less reliable supervision for task selection. The remaining four parameters are comparatively stable around the selected settings, although extreme values can still reduce performance.

Evaluation Integrity. Table 3 examines whether the apparent baseline advantage on ZebraLogic reflects stronger reasoning or evaluation leakage. Panel (a) disentangles access to the gold grid from the executor grammar. The reference condition reproduces the original result, while removing the gold grid reduces performance by 62 points and changing the grammar moves it by at most 5.7. The contrast locates the apparent advantage in prompt leakage rather than in the executor format or stronger reasoning. Panel (b) applies the corrected evaluation protocol to PhantomWiki, where all trained methods improve and our method obtains the largest gain, further widening its margin. Because the

leakage originates from a role-specific prompt rather than the benchmark answer key, reliable multi-role self-play evaluation requires auditing all role-conditioned prompts. **App. B** lists the redacted fields.

	Gold Kept	Gold Removed	Δ
(a) ZebraLogic ($n=300$ Subset), R-Zero Round-5 Checkpoint			
Old Grammar	68.0	6.0	−62.0
New Grammar	62.3	6.7	−55.6
(b) PhantomWiki, Same Leak, Every Method			
Base	25.8	25.0	−0.8
GRPO-Zero	35.8	37.5	+1.7
R-Zero	49.2	50.8	+1.6
LURE (Ours)	60.8	65.8	+5.0

Table 3: Evaluation integrity (%). Both panels contrast the same two conditions on the same tasks.

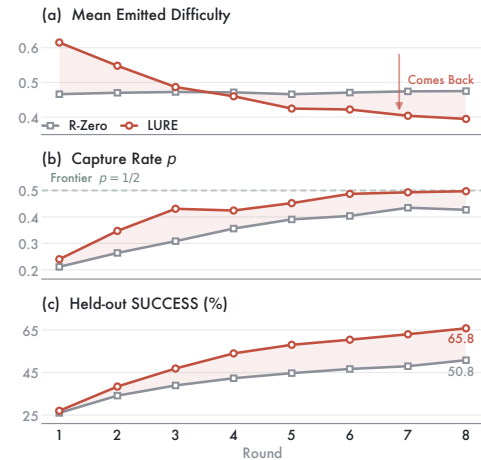


Figure 4: Anatomy of a chase on PhantomWiki per round.

Curriculum Dynamics. Fig. 4 shows how LURE adapts task difficulty to the evolving pursuer on PhantomWiki. At round 1, the capture rate is well below the target frontier

of $p = 0.5$, indicating that the emitted tasks are initially too difficult. **LURE** subsequently lowers the mean difficulty, bringing the capture rate toward the frontier while held-out success increases from approximately 27% to 65.8%. R-Zero instead maintains an approximately fixed emitted difficulty and achieves lower final capture and held-out performance. Thus, **LURE** maintains tasks near a fixed relative difficulty with respect to the current pursuer, rather than increasing difficulty according to a predefined schedule.

Curriculum Cost. Table 4 compares the rollout costs of post-hoc filtering and adaptive task placement. Both methods emit 144 candidates per round. R-Zero spends 1152 probe rollouts to retain 36 tasks and discards the remaining 108 before training, whereas **LURE** eliminates probing and trains on all 144 tasks. Consequently, **LURE** trains on four times as many tasks at one-fifth the rollout cost per task, while reducing the total per-round cost from 1440 to 1152 rollouts.

Rollout budget per round	R-Zero	LURE (Ours)
<i>Selecting the curriculum</i>		
Candidates emitted	144	144
Probe rollouts	1152	0
Candidates discarded	108	0
<i>Training on it</i>		
Tasks entering training	36	144
Training rollouts	288	1152
Total rollouts	1440	1152
Rollouts per training task	40.0	8.0

Table 4: Curriculum cost. Per-round rollouts from candidate generation to training.

Scaling Behavior. Fig. 5 evaluates whether **LURE**’s advantage persists as the backbone scales from 7B to 72B based on Qwen2.5-Instruct series. The three environments exhibit distinct scaling patterns. On PhantomWiki, all methods show diminishing gains at larger scales, while **LURE** maintains a substantial margin over the trained baselines. On IFEval, the methods converge toward a similar performance range: Base performs best at 7B, whereas **LURE** surpasses it from 14B onward. On ZebraLogic, **LURE**’s margin increases

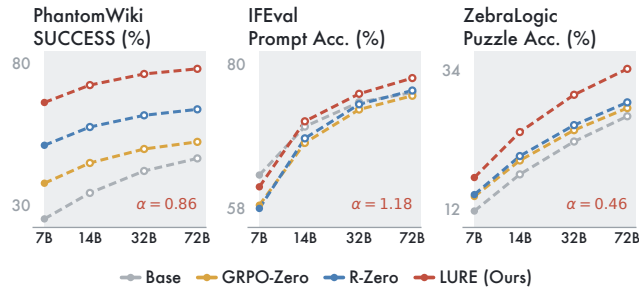


Figure 5: **Scaling behavior** (%) across Qwen2.5-Instruct backbones from 7B to 72B. Filled markers are measured and hollow markers are projections of the power-law fit in App. C, whose exponent α is shown for **LURE**.

OOD Generalization. We further evaluate OOD generalization of the unified model on nine benchmarks across competition mathematics (**MATH500** (Hendrycks et al. 2021; Lightman et al. 2024), **AIME 2024**, **AIME 2025** (Dekoninck et al. 2026), and **OlympiadBench** (He et al. 2024)), rule-based and general reasoning (**KOR-Bench** (Ma et al. 2025), **LiveBench-Reasoning**, and **LiveBench-IF** (White et al. 2024)), and relational or spatial inference (**CLUTRR** (Sinha et al. 2019) and **StepGame** (Shi, Zhang, and Lipani 2022)). Table 5 reports results under a raw single-shot prompt without the planner or executor role, isolating the capabilities retained by the trained backbone, with the per-benchmark protocol in App. B. No method shows consistent gains over Base across individual benchmarks, indicating that the in-domain improvements do not transfer uniformly. At the aggregate level, however, **LURE** is the only trained method to improve over Base, increasing the overall mean by 0.6 points, whereas GRPO and GRPO-Zero decrease by 0.1 and 1.0 points and R-Zero matches Base. Our **LURE** also achieves the highest aggregate score in all three benchmark families, indicating stronger OOD retention despite modest and benchmark-dependent gains.

Benchmark	n	Base	GRPO	GRPO-Zero	R-Zero	LURE
MATH500	100	77.0	78.0	77.0	81.0	79.0
AIME24	30	13.3	13.3	10.0	13.3	16.7
AIME25	30	10.0	10.0	10.0	10.0	10.0
OlympiadBench	300	43.0	42.3	41.7	41.0	41.7
KOR-Bench	250	37.6	37.2	36.8	36.0	39.6
LiveBench-Reasoning	100	26.3	26.0	25.0	25.8	25.0
LiveBench-IF	200	46.1	46.0	45.5	46.5	46.1
CLUTRR	300	32.7	32.7	32.0	33.0	33.7
StepGame	200	20.5	20.5	20.0	20.5	20.5
<i>Mathematics</i>	460	35.8	35.9	34.7	36.3	36.9
<i>Rule-based</i>	550	36.7	36.4	35.8	36.1	36.9
<i>Relational</i>	500	26.6	26.6	26.0	26.8	27.1
Mean		34.1	34.0	33.1	34.1	34.7
Δ vs Base		—	−0.1	−1.0	0.0	+0.6

Table 5: OOD generalization (%) with a raw single-shot prompt and no planner or executor role, on the Qwen2.5-7B-Instruct unified model. **Best** and **second best** per row.

Conclusion

We propose **LURE**, which recasts zero-data self-play as a pursuit-evasion game between a challenger that places tasks and a planner-executor solver that hunts them down. The challenger is trained on a capture-frontier reward that peaks at a capture rate of one half, while the solver receives capture-anchored dense process credit under a round-anchored KL that holds both stable. Across three verifiable reasoning environments and backbone families, **LURE** outperforms baselines under unified and specialist settings and attains stronger aggregate OOD zero-shot accuracy across nine benchmarks, though the gains remain modest and benchmark-dependent.

References

- Bonato, A. 2011. *The game of cops and robbers on graphs*. American Mathematical Soc.
- Dekoninck, J.; Jovanović, N.; Gehringer, T.; Rögnvaldsson, K.; Petrov, I.; Sun, C.; and Vechev, M. 2026. Beyond bench-

- marks: Matharena as an evaluation platform for mathematics with llms. *arXiv preprint arXiv:2605.00674*.
- Dennis, M.; Jaques, N.; Vinitzky, E.; Bayen, A.; Russell, S.; Critch, A.; and Levine, S. 2020. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33: 13049–13061.
- Fang, W.; Liu, S.; Zhou, Y.; Zhang, K.; Zheng, T.; Chen, K.; Song, M.; and Tao, D. 2026. Serl: Self-play reinforcement learning for large language models with limited data. *Advances in Neural Information Processing Systems*, 38: 103706–103738.
- Florensa, C.; Held, D.; Geng, X.; and Abbeel, P. 2017. Automatic goal generation for reinforcement learning agents. *arXiv preprint arXiv:1705.06366*.
- Gong, A.; Wan, C.; Stankevičiūtė, K.; Kabra, A.; Thesmar, R.; Lee, J.; Klenke, J.; Gomes, C. P.; and Weinberger, K. Q. 2025. Phantomwiki: On-demand datasets for reasoning and retrieval evaluation. In *ICML 2025 Workshop on Long-Context Foundation Models*.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- He, C.; Luo, R.; Bai, Y.; Hu, S.; Thai, Z.; Shen, J.; Hu, J.; Han, X.; Huang, Y.; Zhang, Y.; et al. 2024. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3828–3850.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Huang, C.; Yu, W.; Wang, X.; Zhang, H.; Li, Z.; Li, R.; Huang, J.; Mi, H.; and Yu, D. 2025. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*.
- Lambert, N.; Morrison, J.; Pyatkin, V.; Huang, S.; Ivison, H.; Brahman, F.; Miranda, L. J. V.; Liu, A.; Dziri, N.; Lyu, S.; et al. 2024. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, 39578–39601.
- Lin, B. Y.; Le Bras, R.; Richardson, K.; Sabharwal, A.; Poovendran, R.; Clark, P.; and Choi, Y. 2025. Zebralogic: On the scaling limits of llms for logical reasoning. In *Forty-second International Conference on Machine Learning*.
- Liu, B.; Guertler, L.; Yu, S.; Liu, Z.; Qi, P.; Balcells, D.; Liu, M.; Tan, C.; Shi, W.; Lin, M.; et al. 2025. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*.
- Ma, K.; Du, X.; Wang, Y.; Zhang, H.; Wen, Z.; Qu, X.; Yang, J.; Liu, J.; Liu, M.; Yue, X.; et al. 2025. Kor-bench: Benchmarking language models on knowledge-orthogonal reasoning tasks. In *International Conference on Learning Representations*, volume 2025, 80062–80161.
- Prakash, J.; and Buvaresh, A. 2025. What Can You Do When You Have Zero Rewards During RL? *arXiv preprint arXiv:2510.03971*.
- Qwen; ; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; Yang, J.; Tu, J.; Zhang, J.; Yang, J.; Yang, J.; Zhou, J.; Lin, J.; Dang, K.; Lu, K.; Bao, K.; Yang, K.; Yu, L.; Li, M.; Xue, M.; Zhang, P.; Zhu, Q.; Men, R.; Lin, R.; Li, T.; Tang, T.; Xia, T.; Ren, X.; Ren, X.; Fan, Y.; Su, Y.; Zhang, Y.; Wan, Y.; Liu, Y.; Cui, Z.; Zhang, Z.; and Qiu, Z. 2025. Qwen2.5 Technical Report. *arXiv:2412.15115*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*.
- Shi, Z.; Zhang, Q.; and Lipani, A. 2022. Stepgame: A new benchmark for robust multi-hop spatial reasoning in texts. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, 11321–11329.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Sinha, K.; Sodhani, S.; Dong, J.; Pineau, J.; and Hamilton, W. L. 2019. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 4506–4515.
- Sukhbaatar, S.; Lin, Z.; Kostrikov, I.; Synnaeve, G.; Szlam, A.; and Fergus, R. 2017. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*.
- Team, G.; Riviere, M.; Pathak, S.; Sessa, P. G.; Hardin, C.; Bhupatiraju, S.; Hussenot, L.; Mesnard, T.; Shahriari, B.; Ramé, A.; et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.
- Wang, P.; Li, L.; Shao, Z.; Xu, R.; Dai, D.; Li, Y.; Chen, D.; Wu, Y.; and Sui, Z. 2024. Math-shepherd: Verify and

reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9426–9439.

Wang, Y.; Kordi, Y.; Mishra, S.; Liu, A.; Smith, N. A.; Khashabi, D.; and Hajishirzi, H. 2023. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 13484–13508.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.

White, C.; Dooley, S.; Roberts, M.; Pal, A.; Feuer, B.; Jain, S.; Schwartz-Ziv, R.; Jain, N.; Saifullah, K.; Dey, S.; et al. 2024. Livebench: A challenging, contamination-limited llm benchmark. *arXiv preprint arXiv:2406.19314*.

Yao, S.; Zhao, J.; Yu, D.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2022. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*.

Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. 2022. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35: 15476–15488.

Zhao, A.; Wu, Y.; Wu, T.; Xu, Q.; Yue, Y.; Lin, M.; Wang, S.; Wu, Q.; Zheng, Z.; and Huang, G. 2026. Absolute zero: Reinforced self-play reasoning with zero data. *Advances in Neural Information Processing Systems*, 38: 105816–105879.

Zhou, J.; Lu, T.; Mishra, S.; Brahma, S.; Basu, S.; Luan, Y.; Zhou, D.; and Hou, L. 2023. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.